

Normalização de Banco de Dados



O que é normalização de banco de dados?

- A normalização de banco de dados é um processo sistemático usado para organizar dados de forma eficiente em um banco de dados relacional. O objetivo principal da normalização é reduzir a redundância de dados e melhorar a integridade dos dados ao garantir que as dependências dos dados sejam consistentes.
- É indispensável normalizar uma base de dados desde o momento em que ela for desenhada. À medida em que os dados se acumulam e as relações entre eles ficam mais complexas, se torna ainda mais crítico normalizar a base de dados.



Benefícios de normalizar bases de dados!

A normalização de uma base de dados pode oferecer uma série de benefícios importantes:

- **Eliminar a redundância** em uma base de dados previne uma empresa de desperdiçar tempo e recursos. Por exemplo, se os dados de um cliente estão duplicados, isso pode levar ao envio do dobro de materiais de marketing.
- **Na gestão de inventário**, é primordial ter os dados de produtos precisos, atualizados e organizados para evitar a quebra de inventário, a sobre venda e, como consequência, as perdas econômicas e a insatisfação do cliente.
- **Redução de Redundância de Dados**, A normalização elimina dados duplicados, reduzindo o espaço necessário para armazenar informações. Isso torna o banco de dados mais eficiente em termos de armazenamento.



Benefícios de normalizar bases de dados

- **Automatizar operações** em vez de manipular dados manualmente facilita a administração e a análise de dados, tanto quanto reduz custos em recursos humanos, já que os colaboradores se concentram em tarefas mais estratégicas.
- **Melhoria no Atendimento ao Cliente:** Ao centralizar e organizar corretamente as informações dos clientes, a normalização permite que as equipes de suporte acessem rapidamente um histórico completo e preciso, melhorando a qualidade e a velocidade do atendimento.

Esses exemplos demonstram como a normalização não apenas melhora a eficiência técnica do banco de dados, mas também traz benefícios tangíveis para as operações diárias e a estratégia geral de uma organização.



Tipos de anomalias de dados

As anomalias de dados são problemas que podem ocorrer em um banco de dados devido a um design inadequado, especialmente em tabelas que não estão devidamente normalizadas. Essas anomalias podem resultar em inconsistências e dificuldades na gestão dos dados. Existem três tipos principais de anomalias de dados:

- **Anomalias de inserção:** Aqui é quando falta uma informação prévia para adicionar outra, como adicionar um pedido, mesmo sem ter o nome do cliente. Então, para quem será entregue?
- **Anomalias de atualização:** Este tipo é quando alguém tenta arrumar um dado duplicado, mas acaba criando informações conflitantes dentro do banco de dados SQL.
- **Anomalias de exclusão:** O simples deletar de um registro pode levar a uma perda em cadeia de outras informações e inviabilizar o acesso a um cadastro, por exemplo.



Como Evitar Anomalias de Dados

- A melhor maneira de evitar anomalias de dados é projetar o banco de dados seguindo as regras de normalização. Ao aplicar as formas normais adequadas, é possível estruturar os dados de maneira que cada tipo de informação seja armazenado em um local específico, evitando redundâncias e promovendo a consistência e integridade dos dados.



Formas Normais

A normalização é dividida em diferentes etapas, chamadas de formas normais, cada uma com um conjunto específico de regras:

- Primeira forma normal (1FN);
- Segunda forma normal (2FN);
- Terceira forma normal (3FN);



Primeira forma normal (1FN)

- **Critério:** Uma tabela está na 1FN se todos os seus atributos contêm valores atômicos (indivisíveis) ou seja, a tabela não deve conter grupos repetidos e nem atributos com mais de um valor.
- **Exemplo:** Uma tabela de clientes onde cada cliente tenha apenas um número de telefone por registro. Se um cliente tem vários números, esses números seriam armazenados em linhas diferentes.



Exemplo: Tabela de Clientes Antes da 1FN

Suponha que temos uma tabela de clientes onde armazenamos os números de telefone em um único registro por cliente, mas permitimos múltiplos números de telefone no mesmo campo:

Id	Nome	Telefones
1	Beltrano Lima	(11) 1234-56789, (11) 21233-5256
2	Ciclano da Silva	(21) 1236-9999
3	Fulano de Tal	(47) 1233-3222, (11) 5455-5555

Problema: Nesta tabela, o campo "Telefones" não contém valores atômicos, já que pode armazenar múltiplos números separados por vírgulas. Isso viola a 1FN, que exige que todos os valores em uma tabela sejam atômicos (indivisíveis).



Aplicando a 1FN: Para normalizar essa tabela e cumprir a 1FN, devemos armazenar cada número de telefone em uma linha separada, criando um registro distinto para cada combinação de cliente e número de telefone:

Id	Nome
1	Beltrano Lima
2	Ciclano da Silva
3	Fulano de Tal

Cliente_id	Telefone
1	(11) 1234-56789
1	(11) 21233-5256
2	(21) 1236-9999
3	(47) 1233-3222
3	(11) 5455-5555

Resultado: Agora, cada campo armazena apenas um valor atômico, e a tabela está na Primeira Forma Normal (1FN). Isso elimina a necessidade de lidar com valores múltiplos dentro de um único campo e facilita operações como pesquisa, filtragem e atualizações dos números de telefone.



Segunda forma normal (2FN)

- **Critério:** Uma tabela está na 2FN se estiver na 1FN e todos os atributos não-chave forem totalmente dependentes da chave primária inteira (não pode haver dependências parciais).
- **Exemplo:** Em uma tabela de "Pedidos", se a chave primária for composta por "Número do Pedido" e "ID do Produto", os atributos como "Nome do Produto" ou "Preço" devem depender de ambos os elementos da chave primária e não apenas de um deles.



Exemplo: Tabela de Pedidos Antes da 2FN

- Suponha que temos uma tabela "Pedidos" com a seguinte estrutura:

Número do Pedido	ID do Produto	Nome do Produto	Preço	Quantidade
101	A001	Camiseta	25.00	2
101	A002	Calça	40.00	1
102	A001	Camiseta	25.00	1
103	A003	Jaqueta	75.00	3

- **Problema:**

- A chave primária aqui é composta por "Número do Pedido" e "ID do Produto".
- No entanto, os atributos "Nome do Produto" e "Preço" dependem apenas do "ID do Produto" e não do "Número do Pedido".
- Isso caracteriza uma dependência parcial, o que significa que a tabela não está na 2FN.



- **Aplicando a 2FN:** Para normalizar a tabela e colocá-la na Segunda Forma Normal, precisamos separar os atributos que dependem parcialmente da chave primária em uma nova tabela.
- Tabela de Pedidos (Agora na 2FN):

Número do Pedido	Id do Produto	Quantidade
101	A001	2
101	A002	1
102	A001	1
103	A003	3

Tabela de Pedidos: Contém o "Número do Pedido", "ID do Produto" e "Quantidade". Todos os atributos não-chave (neste caso, apenas "Quantidade") dependem da chave primária composta.

- Tabela de Produtos (Nova tabela para eliminar a dependência parcial):

Id do Produto	Nome do Produto	Preço
A001	Camiseta	25.00
A002	Calça	40.00
A003	Jaqueta	75.00

Tabela de Produtos: Contém o "ID do Produto", "Nome do Produto" e "Preço", onde o "Nome do Produto" e "Preço" dependem apenas do "ID do Produto".



- **Benefício:**

- Isso elimina a dependência parcial e garante que cada atributo não-chave seja totalmente dependente da chave primária inteira, colocando as tabelas na Segunda Forma Normal (2FN).
- Essa organização melhora a integridade dos dados e facilita futuras manutenções no banco de dados.



Terceira forma normal (3FN)

- **Critério:** Uma tabela está na 3FN se estiver na 2FN e todos os atributos não-chave forem dependentes diretamente da chave primária (não pode haver dependências transitivas).
- **Exemplo:** Se uma tabela "Funcionários" contém "ID do Funcionário" como chave primária e atributos como "Nome do Funcionário" e "Departamento", e se "Departamento" depende de "Nome do Funcionário", a dependência deve ser eliminada. Em vez disso, deve haver uma tabela separada para "Departamentos" que se relaciona com a tabela "Funcionários".



Exemplo: Tabela de Funcionários Antes da 3FN

- Suponha que temos uma tabela "Funcionários" com a seguinte estrutura:

Id do Funcionário	Nome do Funcionário	Departamento	Nome do Gerente
1	Fulano de Tal	Vendas	Ana Pereira
2	Juana Silva	TI	Carlos Lima
3	Beltrano de Tal	Vendas	Ana Pereira
4	Ciclano Pereira	TI	Carlos Lima

- **Problema:**

- A chave primária aqui é "ID do Funcionário".
- Os atributos "Nome do Gerente" e "Departamento" não dependem diretamente do "ID do Funcionário". O "Nome do Gerente" depende do "Departamento", criando uma **dependência transitiva** (o nome do gerente está relacionado ao departamento, que está relacionado ao funcionário).



- **Aplicando a 3FN:** Para normalizar a tabela e colocá-la na Terceira Forma Normal, precisamos eliminar a dependência transitiva, criando uma nova tabela para os departamentos.
- **Tabela de Funcionários** (Agora na 3FN):

ID do Funcionário	Nome do Funcionário	ID do Departamento
1	Fulano de Tal	101
2	Juana Silva	102
3	Beltrano de Tal	101
4	Ciclano Pereira	102

- **Tabela de Departamentos** (Nova tabela para eliminar a dependência transitiva):

Id do Departamento	Nome do Departamento	Nome do Gerente
1	Vendas	Ana Pereira
2	TI	Carlos Lima



Resultado:

- Agora temos duas tabelas:
- **Tabela de Funcionários:** Contém "ID do Funcionário", "Nome do Funcionário" e "ID do Departamento". Todos os atributos não-chave dependem diretamente da chave primária "ID do Funcionário".
- **Tabela de Departamentos:** Contém "ID do Departamento", "Nome do Departamento" e "Nome do Gerente", onde "Nome do Gerente" e "Nome do Departamento" dependem diretamente do "ID do Departamento".

Benefício:

Isso elimina a dependência transitiva e garante que cada atributo não-chave seja dependente diretamente da chave primária, colocando as tabelas na Terceira Forma Normal (3FN).

Essa estrutura melhora a integridade dos dados, reduz redundâncias e facilita a manutenção e consultas no banco de dados.

